

Programming Techniques 2026–2027

Lecture 3: Basic I/O, Arrays, and Data Type Precision

Hannu Parviainen

Universidad de la Laguna

September 30, 2026

Lecture 2 Recap

Recap: declarations

- ▶ Five intrinsic types: integer, real, complex, logical, and character.
- ▶ Always write `implicit none`, right after any use statements.
- ▶ Declarations come before the executable statements:
`type[, attributes] :: names [= value]`
- ▶ A variable can be given an initial value in its declaration.
- ▶ The parameter attribute makes a **named constant**. Its value is fixed at compile time, and it can be used in later declarations.

```
program declarations
  implicit none
  real, parameter :: pi = 3.14159
  real, parameter :: two_pi = 2 * pi
  integer :: n_planets = 8
  real :: mass, radius = 1.0
  logical :: converged = .false.
  character(len=12) :: star_name = 'Vega'
  complex :: visibility
end program declarations
```

Recap: expressions and assignment

- ▶ Arithmetic: `**`, `*`, `/`, `+`, `-`
- ▶ Relational: `<`, `<=`, `==`, `/=`, `>=`, `>`
“Not equal” is `/=`, because `!` starts a comment.
- ▶ Logical: `.not.`, `.and.`, `.or.`, `.eqv.`, `.neqv.`
- ▶ Assignment evaluates the right-hand side first, then silently converts it to the type of the left-hand side.
- ▶ Do not compare reals with `==`, but compare within a tolerance instead.

```
program expressions
  implicit none
  integer :: n_stars
  real :: m1 = 1.0, m2 = 1.00001
  logical :: equal_mass
  n_stars = 3.7 ! truncated to 3, -Wall warns
  equal_mass = abs(m1 - m2) < 1.0e-4
  print *, n_stars, equal_mass, m1 /= m2
end program expressions
```

Recap: conditional execution

- ▶ One-line if: runs a single statement when the condition is true.
- ▶ Block if: else if and else are both optional.
- ▶ select case chooses between single values, such as (0), and ranges, such as (2:7) or (2:). An optional case default catches everything else.

```
program conditionals
  implicit none
  real :: e = 0.3
  integer :: n_planets = 3
  if (e < 0.0) print *, 'Invalid eccentricity'
  if (e < 1.0) then
    print *, 'Elliptical orbit (bound)'
  else if (e > 1.0) then
    print *, 'Hyperbolic orbit (unbound)'
  else
    print *, 'Parabolic orbit'
  end if
  select case (n_planets)
  case (0)
    print *, 'No known planets'
  case (2:)
    print *, 'Multi-planet system'
  end select
end program conditionals
```

Recap: loops

- ▶ `do i = start, stop [, step]` runs a fixed number of times. The step defaults to 1.
- ▶ `do while (condition)` repeats while the condition is true.
- ▶ A plain `do` repeats until an `exit`.
- ▶ `exit` leaves the loop, and `cycle` skips to its next iteration.
- ▶ Loops can be named, and `exit` and `cycle` can name the loop they act on. Otherwise they act on the innermost loop.

```

program loops
  implicit none
  integer :: i, frame, total
  total = 0
  do i = 1, 9, 2                ! i = 1, 3, 5, 7, 9
    total = total + i
  end do
  do while (total > 1)
    total = total / 2
  end do
  frame = 0
  frames: do
    frame = frame + 1
    if (frame > 100) exit frames
    if (frame >= 50 .and. frame <= 59) cycle frames
    print *, 'Reducing frame', frame
  end do frames
end program loops

```

Basic Input and Output

Output: print and write

- ▶ `print *`, `items` and `write(*,*)` items both write a line to standard output, normally the terminal.
- ▶ In `write(*,*)`, the first `*` is the unit (standard output) and the second is the format. A `*` format lets the compiler choose the layout (*list-directed* output).
- ▶ Each statement writes one line, and `print *` alone writes an empty line.
- ▶ Formats for controlling the layout come later in the course.

```
program output
  implicit none
  real :: radius = 6371.0
  integer :: n_moons = 1
  print *, 'Earth radius:', radius, 'km'
  write(*,*) 'Moons:', n_moons, 'twice:', 2*n_moons
  print *
  print *, 'Is Earth a planet?', .true.
end program output
```

```
$ gfortran output.f90 && ./a.out
Earth radius:   6371.00000          km
Moons:          1 twice:           2

Is Earth a planet? T
```

Input: read

- ▶ `read *, items` and `read(*,*)`
items read from standard input, normally the keyboard.
- ▶ The values can be separated by spaces, commas, or line breaks. If a line runs out, `read` continues on the next one.
- ▶ Each `read` starts on a new line, and values left over on the current line are skipped.
- ▶ Invalid input, such as text where a number is expected, stops the program with a runtime error.

```
program density
  implicit none
  real, parameter :: pi = 3.14159
  real :: mass, radius, rho
  print *, 'Mass (kg) and radius (m)?'
  read *, mass, radius
  rho = mass / (4.0/3.0 * pi * radius**3)
  print *, 'Mean density (kg/m^3):', rho
end program density
```

```
$ gfortran density.f90 && ./a.out
Mass (kg) and radius (m)?
5.97e24 6.371e6
Mean density (kg/m^3): 5511.41650
```

Reading until the end of input

- ▶ With `iostat=ios`, `read` reports a status instead of stopping: 0 is success, negative is the end of the input, and positive is an error.
- ▶ This lets a program read any number of values.
- ▶ The shell can connect standard input and output to files or other programs, with no change to the code:
`./a.out < mags.txt,`
`./a.out > out.txt, or`
`prog1 | ./a.out.`

```

program mean
  implicit none
  real :: x, total = 0.0
  integer :: n = 0, ios
  do
    read(*, *, iostat=ios) x
    if (ios > 0) print *, 'Not a number, stopping'
    if (ios /= 0) exit
    total = total + x
    n = n + 1
  end do
  if (n > 0) print *, n, 'values, mean =', total / n
end program mean

```

```

$ ./a.out < mags.txt      # mags.txt: 4.2, 3.9, 4.5
                        3 values, mean = 4.20000029

```

Catching input errors

- ▶ `iomsg=msg` gives the error message.
- ▶ After a failed read, the input position is undefined. So read the whole line as text ('(a)'), then convert it with an *internal read* from the string.
- ▶ `read` does not check that values make sense. Test ranges yourself.

```
Give a number:
abc
Bad real number in item 1 of list input
Give a number:
2.5
Got 2.50000000
```

```
program read_number
  implicit none
  real :: x
  integer :: ios
  character(len=100) :: line
  character(len=200) :: msg
  do
    print *, 'Give a number:'
    read(*, '(a)', iostat=ios) line
    if (ios /= 0) stop 'No input'
    read(line, *, iostat=ios, iomsg=msg) x
    if (ios == 0) exit
    print *, trim(msg)
  end do
  print *, 'Got', x
end program read_number
```

Arrays

Array Terminology

Definitions

- ▶ **Rank:** number of dimensions (e.g., 1D, 2D).
- ▶ **Bounds:** lower/upper index limits in each dimension.
- ▶ **Extent:** number of elements along a dimension.
- ▶ **Size:** total number of elements.
- ▶ **Shape:** tuple of extents.
- ▶ **Conformable:** same shape (element-wise operations valid).

Explicit-Shape Declarations

Forms

- ▶ `real, dimension(100) :: R`
- ▶ `real, dimension(1:10,1:10) :: S`
- ▶ `real :: T(10,10)`
- ▶ `real, dimension(-10:-1) :: X`
- ▶ `integer, parameter :: lda = 5`
`real, dimension(0:lda-1) :: Y`
- ▶ `real, dimension(1+lda*lda,10) :: Z`

Notes

Default lower bound is 1; bounds may begin/end anywhere; arrays can be zero-sized if bounds imply size 0.

Conformance Rules

Element-wise operations

Arrays (or sections) in an expression or assignment must conform: they must have the same shape. A scalar conforms with any array.

Examples

```
real :: A(3,4), B(4,3), C(3,4), D(3,4)
C = 1.0    ! valid: the scalar is copied to every element
C = D      ! valid: same shape, [3,4]
B = A      ! invalid: same size (12), but different shape
```

Checking

Here the shapes are known when compiling, so gfortran rejects the last line. When shapes are only known at run time, compile with `-fcheck=all`.

Array Element Order

- ▶ Fortran uses **column-major** order: the first index varies fastest, so the elements of each column are next to each other in memory.
- ▶ The same order is used by `print`, `read`, and `reshape`.
- ▶ C and NumPy use the opposite, *row-major*, order by default.

```
real :: A(3,4)
```

A(1,1)	A(1,2)	A(1,3)	A(1,4)
A(2,1)	A(2,2)	A(2,3)	A(2,4)
A(3,1)	A(3,2)	A(3,3)	A(3,4)

memory:

A(1,1)	A(2,1)	A(3,1)	A(1,2)	A(2,2)	A(3,2)	A(1,3)	A(2,3)	A(3,3)	A(1,4)	A(2,4)	A(3,4)
1	2	3	4	5	6	7	8	9	10	11	12

Loop Order Matters

- ▶ The CPU reads memory in blocks (cache lines), so a loop is fastest when consecutive iterations use neighbouring elements.
- ▶ In Fortran, put the **first** index in the **innermost** loop.
- ▶ Compiled with -O2, the row-by-row loop here is about three times slower, and the gap grows with the array size.
- ▶ In C and NumPy, it is the other way round.

```
program loop_order
  implicit none
  integer, parameter :: n = 5000
  integer :: a(n,n), s1 = 0, s2 = 0, i, j
  real :: t0, t1, t2
  a = 1
  call cpu_time(t0)
  do j = 1, n          ! fast: first index in the inner loop
    do i = 1, n
      s1 = s1 + a(i,j)
    end do
  end do
  call cpu_time(t1)
  do i = 1, n          ! slow: each step jumps n elements
    do j = 1, n
      s2 = s2 + a(i,j)
    end do
  end do
  call cpu_time(t2)
  print *, t1 - t0, t2 - t1, s1 == s2 ! ~0.005 s, ~0.015 s, T
end program loop_order
```

Subscript Triplets

General form

[lb] : [ub] [:stride] with scalar integer expressions.

Examples

```
A(:)      ! whole array
A(3:9)    ! A(3) .. A(9) step 1
A(3:9:1)   ! same as above
A(m:n)    ! lower/upper bounds
A(m:n:k)   ! step k
A(8:3:-1)  ! descending section
A(8:3)     ! zero-sized (default step 1)
A(m:)     ! m .. upper bound
A(:n)     ! lower bound .. n
A(::2)    ! every 2nd element
A(m:m)    ! 1-element section
A(m)      ! scalar element (not a section)
```

Array Indexing: Single Element

Indexing Example

`A(3,4)`

Selects one element at row 3, column 4.

Declaration

```
real :: A(5,5)
```

Result

A scalar.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Row Section

Indexing Example

`A(2,:)`

Selects all columns on row 2.

Declaration

`real :: A(5,5)`

Result

A rank-1 array of size 5. The scalar subscript removes a dimension: `A(2:2,:)` would keep it, with shape `[1,5]`.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Column Section

Indexing Example

`A(:,3)`

Selects all rows in column 3.

Declaration

`real :: A(5,5)`

Result

A rank-1 array of size 5.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Subarray

Indexing Example

`A(2:4, 2:5)`

Selects rows 2–4 and columns 2–5 (a 3x4 block).

Declaration

```
real :: A(5,5)
```

Result

A rank-2 array of shape [3,4].

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Strided Selection

Indexing Example

`A(1:5:2, 2:5:2)`

Selects odd rows and even columns.

Declaration

`real :: A(5,5)`

Result

A rank-2 array of shape `[3,2]`: rows 1, 3, 5 and columns 2, 4.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Vector Subscripts

- ▶ An integer array used as a subscript selects the listed elements, in any order and possibly more than once.
- ▶ Useful for picking entries by index, e.g. selected stars from a catalogue.
- ▶ A vector subscript can also be used on the left-hand side of an assignment, but then no index may repeat.

```
program vector_subscripts
  implicit none
  real :: x(6) = [10., 20., 30., 40., 50., 60.]
  integer :: idx(3) = [5, 1, 3]
  print *, x(idx)           ! 50. 10. 30.
  print *, x([2, 2, 6])     ! 20. 20. 60.
  x(idx) = 0.0              ! set the selected elements
  print *, x                ! 0. 20. 0. 40. 0. 60.
end program vector_subscripts
```

Whole Arrays, Elements, and Sections

Declarations

```
real :: A(5), B(-1:3,3), C(5,3), D(5,3)
```

Whole arrays

```
A = 0.0      ! set every element of A to zero  
B = C + D    ! element-wise add C and D into B
```

Elements

```
A(1) = 0.0  
B(0,1) = A(3) + C(5,1)
```

Sections

```
A(2:4) = 0.0  
B(-1:0,1:2) = C(1:2,2:3) + 1.0
```

Whole-Array Expressions

Elemental semantics

Operators and elemental intrinsic functions (`sin`, `exp`, `abs`, ...) act element by element on conformable arrays.

Examples

```
real :: B(5,3), C(5,3), D(5,3)
B = C * D - B**2      ! element by element
B = sin(C) + cos(D)   ! elemental intrinsics
```

Not matrix multiplication

`C * D` multiplies matching elements. The matrix product is `matmul`, covered with the other array intrinsics below.

Array Assignment Evaluates the Right-Hand Side First

- ▶ In an array assignment, the whole right-hand side is evaluated before anything is stored.
- ▶ So `a(2:5) = a(1:4)` shifts the elements by one, as expected.
- ▶ A do loop that looks the same overwrites each element before it is read, and copies `a(1)` everywhere.

```
program shift
  implicit none
  integer :: i, a(5), b(5)
  a = [1, 2, 3, 4, 5]
  b = a
  a(2:5) = a(1:4)      ! array assignment
  print *, a           ! 1 1 2 3 4
  do i = 2, 5          ! the "same" as a loop
    b(i) = b(i-1)
  end do
  print *, b           ! 1 1 1 1 1
end program shift
```

Array Intrinsics: Inquiry and Reductions

- ▶ Inquiry: size, shape, lbound, ubound.
- ▶ Reductions: sum, product, minval, maxval, and the locations minloc, maxloc.
- ▶ With dim=, a reduction works along one dimension: sum(m, dim=1) sums each column.
- ▶ Logical reductions take a mask: count, any, all.

```
program intrinsics
  implicit none
  real :: m(2,3)
  m = reshape([1., 2., 3., 4., 5., 6.], [2, 3])
  ! m = | 1 3 5 |
  !     | 2 4 6 |

  print *, size(m), shape(m)           ! 6  2 3
  print *, lbound(m), ubound(m)        ! 1 1  2 3
  print *, sum(m), sum(m, dim=1)        ! 21.  3. 7. 11.
  print *, maxval(m), maxloc(m)         ! 6.  2 3
  print *, count(m > 3.0)               ! 3
  print *, any(m < 0.0), all(m > 0.0)   ! F T
end program intrinsics
```

Array Intrinsics: Matrices and Vectors

- ▶ `matmul(a, b)`: the matrix product.
The inner extents must agree:
 $[2, 3] \times [3, 2]$ gives $[2, 2]$.
- ▶ `dot_product(u, v)`: the scalar product of two vectors.
- ▶ `transpose(a)`: swaps rows and columns.
- ▶ `reshape(source, shape)`: fills a new shape in column-major order.

```
program linalg
  implicit none
  real :: a(2,3), b(3,2), c(2,2), u(3) = [1., 2., 3.]
  a = reshape([1., 2., 3., 4., 5., 6.], [2, 3])
  b = transpose(a)
  c = matmul(a, b)
  print *, dot_product(u, u)      ! 14.
  print *, c                      ! 35. 44. 44. 56.
end program linalg
```

Masked Assignment: where

- ▶ `where` assigns only to the elements where a logical mask is true. `elsewhere` handles the rest.
- ▶ Here, magnitudes are computed only for positive fluxes (`log10` is never called for the others), and the rest are flagged with 99.
- ▶ Short form for a single assignment:
`where (flux < 0.0) flux = 0.0`

```
program masked
  implicit none
  real :: flux(6) = [3.2, -1.0, 5.1, 0.0, -0.5, 2.4]
  real :: mag(6)
  where (flux > 0.0)
    mag = -2.5 * log10(flux)
  elsewhere
    mag = 99.0
  end where
  print *, mag      ! -1.26 99. -1.77 99. 99. -0.95
end program masked
```

Array I/O Ordering

```
print *, A
```

Writes all elements on one line, in column-major order: $A(1,1)$, $A(2,1)$, and so on.

```
read *, A
```

Reads the elements in the same order. A matrix stored row by row in a file must be read one row at a time, or transposed after reading.

Printing a matrix row by row

```
do i = 1, size(A, 1)
  print *, A(i,:)
end do
```

Array I/O Example

Program

```
program ex3
  implicit none
  integer, parameter :: n = 3
  integer :: a(n,n), i
  a = reshape([1, 2, 3, 4, 5, 6, 7, 8, 9], [n, n])
  print *, 'Element   =', a(3,2)
  print *, 'Column 1   =', a(:,1)
  print *, 'Subarray   =', a(:2,:2)
  print *, 'Whole      =', a
  print *, 'Transposed =', transpose(a)
  do i = 1, n
    print *, a(i,:)
  end do
end program ex3
```

Predict the output before you run it.

Array I/O Example

Program

```
program ex3
  implicit none
  integer, parameter :: n = 3
  integer :: a(n,n), i
  a = reshape([1, 2, 3, 4, 5, 6, 7, 8, 9], [n, n])
  print *, 'Element   =', a(3,2)
  print *, 'Column 1   =', a(:,1)
  print *, 'Subarray   =', a(:2,:2)
  print *, 'Whole      =', a
  print *, 'Transposed =', transpose(a)
  do i = 1, n
    print *, a(i,:)
  end do
end program ex3
```

Predict the output before you run it.

Output (spacing condensed)

```
Element   = 6
Column 1   = 1 2 3
Subarray   = 1 2 4 5
Whole      = 1 2 3 4 5 6 7 8 9
Transposed = 1 4 7 2 5 8 3 6 9
1 4 7
2 5 8
3 6 9
```

Catching Array Errors: -fcheck=all

- ▶ By default, Fortran does not check subscripts. Writing `x(6)` in a 5-element array silently overwrites other memory: the program may crash or, worse, run and give wrong results.
- ▶ Compile with `-g -fcheck=all` during development. The program then stops at the first bad subscript and says where. It also checks at run time that array shapes conform.
- ▶ The checks slow the program down, so leave them out of production runs.

```
program out_of_bounds
  implicit none
  integer :: i, n
  real :: x(5)
  n = 6
  do i = 1, n
    x(i) = i      ! x(6) does not exist
  end do
  print *, x
end program out_of_bounds
```

```
$ gfortran -g -fcheck=all out_of_bounds.f90
$ ./a.out
At line 7 of file out_of_bounds.f90
Fortran runtime error: Index '6' of dimension 1 of array '
  x' above upper bound of 5
```

Data Type Precision

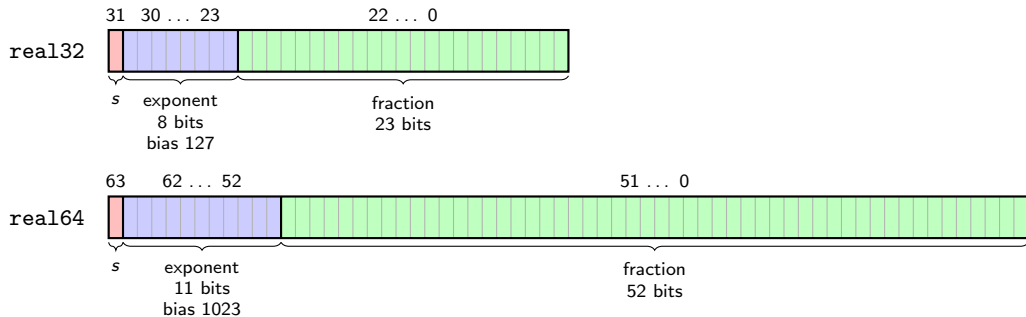
Floating-point numbers

- ▶ Computers store reals in binary scientific notation, like 6.022×10^{23} but in base 2:

$$x = (-1)^s \times 1.f \times 2^{e-\text{bias}}$$

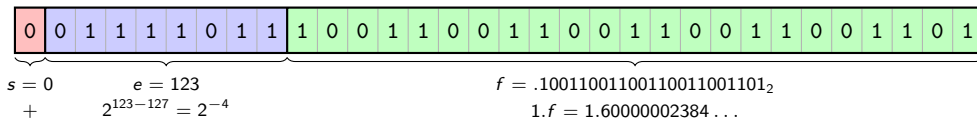
- ▶ A real has a fixed number of bits, split into three fields: the sign s , the exponent e , and the fraction f .
- ▶ The leading 1 is implied and not stored. The bias lets the stored exponent be a non-negative integer.
- ▶ The fixed number of bits sets two limits:
 - ▶ the fraction bits set the **precision**: how many significant digits a number has,
 - ▶ the exponent bits set the **range**: how large and how small a number can be.

Bit layout of 32- and 64-bit reals



- ▶ 32-bit real (*single precision*): about 7 significant digits, range about $10^{\pm 38}$.
- ▶ 64-bit real (*double precision*): about 16 significant digits, range about $10^{\pm 308}$.
- ▶ In Fortran: `real(real32)` and `real(real64)`, explained at the end of this section.

Example: 0.1 as a 32-bit real



- ▶ $0.1 = 1.6 \times 2^{-4}$, so $s = 0$ and $e = -4 + 127 = 123$.
- ▶ In binary, $0.6 = 0.1001\ 1001\ 1001 \dots_2$ repeats forever, so the fraction must be rounded to 23 bits.
- ▶ The stored value is $0.100000001490116 \dots$

Most decimals are not exact

Just as $1/3 = 0.333 \dots$ has no finite decimal form, 0.1 has no finite binary form. Most numbers are rounded when they are stored, and this rounding is where precision starts to matter.

How precision matters: the gaps grow with magnitude

- ▶ Reals are not evenly spaced. With a fixed number of significant digits, the gap to the next real grows with the size of the number (`spacing(x)`).
- ▶ Example: a Julian date, $JD = 2460000.5$. A 32-bit real can only store it in steps of 0.25 days, that is, **6 hours**. A 64-bit real resolves $40\ \mu\text{s}$.
- ▶ Fix: keep the numbers small, e.g. store times relative to a reference epoch, $t - t_0$.

```
program jd_precision
  use iso_fortran_env, only: real32, real64
  implicit none
  real(real32) :: jd_s = 2460000.5
  real(real64) :: jd_d = 2460000.5_real64
  print *, spacing(jd_s)           ! 0.250000000 days
  print *, spacing(jd_d) * 86400 ! 4.02E-05 seconds
end program jd_precision
```

How precision matters: rounding errors add up

- ▶ Every operation rounds its result, and the rounding errors add up.
- ▶ Adding 0.1 ten million times should give 10^6 . With 32-bit reals, the result is off by 9%.
- ▶ The same happens with $t = t + dt$ in a time integration.
- ▶ Fix: compute $t = i*dt$ instead, or use 64-bit reals.

```
program accumulate
  use iso_fortran_env, only: real32, real64
  implicit none
  integer :: i
  real(real32) :: s32 = 0.0
  real(real64) :: s64 = 0.0_real64
  do i = 1, 10000000
    s32 = s32 + 0.1_real32
    s64 = s64 + 0.1_real64
  end do
  print *, s32      ! 1087937.00
  print *, s64      ! 999999.99983897537
end program accumulate
```

How precision matters: subtraction cancels digits

- ▶ Subtracting two nearly equal numbers cancels their leading digits. What is left is mostly rounding error.
- ▶ Example: for $x = 10^{-4}$, $\cos x$ rounds to exactly 1.0 in 32-bit, so $1 - \cos x$ gives 0 instead of 5×10^{-9} .
- ▶ Fix: rewrite the formula. The equivalent $2\sin^2(x/2)$ has no subtraction and is accurate even in 32-bit.

```
program cancellation
  implicit none
  real :: x = 1.0e-4
  print *, 1.0 - cos(x)           ! 0.00000000
  print *, 2.0 * sin(x/2)**2     ! 4.99999953E-09
end program cancellation
```

How precision matters: the range is limited

- ▶ Results too large for the type become Infinity (*overflow*). The solar mass squared in kg^2 , about 4×10^{60} , overflows a 32-bit real.
- ▶ Results too close to zero become 0 (*underflow*).
- ▶ Integers overflow without any warning: 50000^2 wraps around to a negative number in a 32-bit integer.
- ▶ Fix: use 64-bit types, or units that keep numbers moderate ($M_\odot$, au, days).

```
program range_limits
  use iso_fortran_env, only: real32, int32
  implicit none
  real(real32) :: m_sun = 1.989e30 ! kg
  real(real32) :: x = -104.0
  integer(int32) :: n = 50000
  print *, m_sun**2 ! Infinity
  print *, exp(x) ! 0.00000000
  print *, n*n ! -1794967296
end program range_limits
```

Choosing the precision

Use 64-bit reals by default

About 16 significant digits and a range of $10^{\pm 308}$ are enough for most scientific computing, at a small cost.

32-bit reals have their place

They halve the memory use and are faster, especially on GPUs. Use them for large data sets when 7 digits are enough.

More bits do not fix bad numerics

Keep numbers moderate (scaled units, $t - t_0$), rewrite formulas that cancel, and never compare reals with `==`.

Kind parameters

- ▶ In Fortran, the precision of a type is set by its **kind** parameter: `integer(kind=k)`, `real(kind=k)`, or simply `real(k)`.
- ▶ The kind is an integer, but the standard does not say what the numbers mean. Each compiler chooses its own:
 - ▶ GNU Fortran: the kind is the number of bytes, so `real(8)` is a 64-bit real.
 - ▶ NAG Fortran (by default): kinds 1 and 2 are the 32- and 64-bit reals, and `real(8)` does not exist.
- ▶ Hard-coded kind numbers are therefore not portable. The next slides show three portable alternatives.
- ▶ C has a similar problem: the size of `int` and `long` depends on the platform.

Kind numbers in GNU Fortran

- ▶ The kind is the number of bytes:
 - ▶ integer and logical: 1, 2, 4, 8, 16
 - ▶ real and complex: 4, 8, 10, 16
- ▶ A complex number holds two reals, so `complex(4)` takes 8 bytes.
- ▶ The default kind is 4 for integer, logical, real, and complex, and 1 for character. `double precision` is `real(8)`.
- ▶ Fine for quick tests, but prefer the portable methods that follow.

```
real(kind=4)      :: a  ! 32-bit real
real(kind=8)      :: b  ! 64-bit real
integer(kind=4)   :: i  ! 32-bit integer
integer(kind=8)   :: j  ! 64-bit integer
```

Portable kinds by precision and range

- ▶ State what you need, and let the compiler pick the kind:
 - ▶ `selected_int_kind(r)`: the smallest integer kind that holds all n with $-10^r < n < 10^r$.
 - ▶ `selected_real_kind(p, r)`: a real kind with at least p significant digits and an exponent range of at least $10^{\pm r}$.
- ▶ `selected_real_kind(15, 307)` gives a 64-bit real with any common compiler.
- ▶ If no kind fits, the result is negative and the declaration fails to compile.

```
program ex3a
  implicit none
  integer, parameter :: ik = selected_int_kind(15)
  integer, parameter :: dp = selected_real_kind(15, 307)
  integer(ik) :: n
  real(dp) :: x
  print *, huge(n)                ! 9223372036854775807
  print *, precision(x), range(x) ! 15 307
end program ex3a
```

Portable kinds by size: ISO_FORTRAN_ENV

- ▶ The intrinsic module `iso_fortran_env` names the kinds by their size in bits: `int8`, ..., `int64`, `real32`, `real64`, `real128`.
- ▶ The module is from Fortran 2003, and these names from Fortran 2008.
- ▶ They fix the number of bits, not the precision, but in practice they are the 32- and 64-bit reals shown earlier.
- ▶ Common practice: define the kind once as a named constant (`dp` in the example) and use it everywhere.

```
program ex3b
  use iso_fortran_env, only: int32, int64, real32, real64
  implicit none
  integer, parameter :: dp = real64
  integer(int32) :: i
  integer(int64) :: j
  real(real32) :: x
  real(dp) :: y
  print *, huge(i), huge(j)
  print *, tiny(x), huge(x)
  print *, tiny(y), huge(y)
end program ex3b
```

Trap: literals have a kind too

- ▶ A real literal without a kind, such as 0.1, is a default real, that is, 32-bit.
- ▶ It is rounded to 32 bits before it is converted to 64 bits, so it keeps the 32-bit rounding error we saw for 0.1.
- ▶ Always give real literals a kind: 0.1_dp, 6.674e-11_dp. This includes literals in expressions, such as $x * 0.1$.
- ▶ Older code writes 0.1d0, where the exponent letter d means double precision.

```
program literal_kinds
  use iso_fortran_env, only: real64
  implicit none
  integer, parameter :: dp = real64
  real(dp), parameter :: pi = 3.141592653589793_dp
  real(dp) :: a = 0.1      ! 32-bit literal
  real(dp) :: b = 0.1_dp   ! 64-bit literal
  print *, a      ! 0.10000000149011612
  print *, b      ! 0.10000000000000001
  print *, pi
end program literal_kinds
```

Kinds for C interoperability: ISO_C_BINDING

- ▶ The intrinsic module `iso_c_binding` (Fortran 2003) provides kinds that match the C types: `c_int`, `c_long`, `c_float`, `c_double`, ...
- ▶ Use them when Fortran and C code exchange data, e.g. when calling a C library.
- ▶ On common platforms, `c_float` and `c_double` are the same as `real32` and `real64`.

```
program ex3c
  use iso_c_binding, only: c_float, c_double
  implicit none
  real(c_float) :: x
  real(c_double) :: y
  print *, tiny(x), huge(x)
  print *, tiny(y), huge(y)
end program ex3c
```

Exercises

Exercise 1:

- ▶ Write a program that computes and prints the matrix multiplication of two real arrays.

$$A = \begin{pmatrix} 3 & 2 & 4 & 1 \\ 2 & 4 & 2 & 2 \\ 1 & 2 & 3 & 7 \end{pmatrix} \quad B = \begin{pmatrix} 3 & 2 & 4 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \\ 0 & 2 & 1 \end{pmatrix}$$

Exercise 2:

- ▶ Write a Fortran program that reads an integer from the user and determines whether it is a palindrome (whether its digits read the same forwards and backwards).